

## NOTE DE PROJET

# Ops Agent

Assistant IA personnel modulaire, testé comme du vrai logiciel, construit directement dans le repo `workspace` pour allier montée en compétences, optimisation du quotidien et première monétisation.

---

Préparé pour Steew · 21 juillet 2026

Repo cible : [github.com/steew258-bot/workspace](https://github.com/steew258-bot/workspace)

## 1 Concept & facteur surprise

Plutôt qu'un assistant « un prompt = une réponse », **Ops Agent** est un agent modulaire où chaque tâche du quotidien (triage email/admin, veille d'opportunités, planification) devient un module avec un contrat d'entrée/sortie strict (JSON structuré) et un test automatisé qui vérifie sa fiabilité.

**Ce qui change tout** : la plupart des « assistants IA perso » sont du bricolage non fiable. Ici, chaque prompt est traité comme du code — testé, versionné, validé en CI — ce qui en fait un produit différenciant plutôt qu'un gadget.

Le projet réutilise directement l'infrastructure déjà en place dans le repo (venv Python, ruff, pytest, GitHub Actions, connexion MCP GitHub) au lieu de la laisser tourner à vide.

## 2 Compétence IA développée

- Architecture d'agent modulaire (tool use / sorties structurées avec l'API Claude)
- Prompt chaining avec **eval harness** — tester un prompt comme du code
- Automatisation planifiée (GitHub Actions cron, gratuit)
- Intégration API réelle via le serveur MCP GitHub déjà connecté ( `.mcp.json` )
- Packaging produit — transformer un repo personnel en template vendable

## 3 Stratégie de monétisation

Deux phases séquentielles au sein d'un seul et même projet :

1. **Phase interne (semaines 1-2)** — usage personnel quotidien ; la valeur se mesure en temps récupéré, pas encore en revenu direct.
2. **Phase produit (semaine 3+)** — une fois 2-3 modules fiabilisés et testés, nettoyage du repo (README réel, données perso retirées) puis publication comme template payant (Gumroad / GitHub Template, 19-29€), ou utilisation comme vitrine pour vendre un service d'audit-automatisation à d'autres indépendants.

## 4 Calcul de rendement

| Poste                                         | Temps estimé   | Retour                      |
|-----------------------------------------------|----------------|-----------------------------|
| Socle (structure + module « triage » + test)  | 4 – 6 h        | Réutilise l'infra existante |
| Module supplémentaire (veille, planification) | 2 – 3 h chacun | Squelette réutilisable      |

|                                 |         |                               |
|---------------------------------|---------|-------------------------------|
| Gain quotidien (usage perso)    | —       | 20 – 40 min/jour → ~10 h/mois |
| Packaging & publication produit | 2 – 3 h | Coût marginal quasi nul       |
| Seuil de rentabilité produit    | —       | 1 vente à 20-29€ suffit       |

## 5 Plan d'action pas à pas

---

- 1 Restructurer `app.py` en point d'entrée CLI de l'agent ; créer `src/modules/`
- 2 Construire le module `triage` : texte libre → JSON structuré ( `action` , `urgence` , `brouillon_reponse` ) via l'API Claude
- 3 Écrire un test pytest avec 3-4 cas fixes, branché sur la CI existante
- 4 Ajouter `.github/workflows/veille.yml` (cron quotidien) qui pousse les résultats comme issue GitHub via le MCP déjà configuré
- 5 Utiliser l'agent 10-14 jours, journaliser le temps récupéré
- 6 Compléter le README, retirer les données perso, publier le repo comme template
- 7 Poster dans 1-2 communautés ciblées (freelances, indie hackers) avec une démo courte

---

Ops Agent — note de projet générée le 21 juillet 2026. Ce document synthétise la discussion de cadrage pour le repo `workspace` .